COU4303-Artifical Intelligence
PROJECT REPORT
Level 04

SMART AMBULANCE ROUTE FINDING SYSTEM

Submitted by:
M.F Noora -524656421



Bachelor of Science in Information Technology

Faculty of Natural Sciences

The Open University of Sri Lanka

Table of Contents

COU4303-Artifical Intelligence 1
1. Problem Statement 3
2. Proposed Solution 3
2.1 System Features 4
2.2 How the System Works 4
3. Explanation of Code 5
3.1 Road and Traffic Facts 5
3.2 Blocked Roads and Road availability 5
3.3 Road Cost Calculation 5
3.4 Quick Route - DFS 6
3.5 Shortest Route - BFS 6
3.6 Smart Route – A* 6
3.7 Route Comparison and Main Menu 6
3.8 Sample Output 7
4. Advanced Concepts Used 9
4.1 Search Algorithms 9
4.2 Depth First Search (DFS) 9
4.3 Breadth First Search (BFS) 9
4.4 A* Search 9
4.5 Heuristic Function 9
4.6 Knowledge Representation 9
4.7 Recursion and Backtracking 10
4.8 List Processing 10
4.9 Modular Design 10
5. Testing and Results 11
6. Algorithm Comparison 13
7. Advantages and Limitations 14
7.1 Advantages 14
7.2 Limitations 14
8. Future Enhancements 15
9. Conclusion 15

1. Problem Statement

In emergency situations, ambulances need to reach their destinations quickly. However, finding a suitable route can be difficult when there are multiple roads, different distances, traffic conditions, and blocked roads.

A short-distance road may not always be the best option because it may have heavy traffic. Also, some roads may be unavailable because of road blocks. Therefore, an intelligent system is needed to help find suitable routes for ambulances.

The system should be capable of:

- Finding a route between two locations
- Avoiding blocked roads
- Considering road distances
- Considering traffic conditions
- Finding a quick route
- Finding a shortest route
- Finding a smart route
- Comparing different route options

To address this problem, this project proposes a Prolog-based Smart Ambulance Route Finding System. The system uses Artificial Intelligence search techniques such as Depth First Search (DFS), Breadth First Search (BFS), and A* Search to find different types of routes.

2. Proposed Solution

The Smart Ambulance Route Finding System is a Prolog-based artificial Intelligence system that helps find suitable routes between locations.

The system stores road information including distance and estimated travel time. It also stores traffic conditions and blocked roads.

The system provides three route-finding methods:

- Quick Route – uses DFS
- Shortest Route – uses BFS
- Smart Route – uses A*

The system also provides options to check road status, display traffic information, and compare route options.

The main components are:

- Road facts
- Traffic facts
- Blocked road facts
- Road availability checking
- Road cost calculation
- DFS search
- BFS search
- A* search
- Route comparison
- User menu

2.1 System Features

- The main features of the system are:
- Finds ambulance routes between locations
- Provides quick route
- Provide shortest route
- Provide smart route
- Avoid blocked roads
- Stores road distance and travel time
- Considers traffic levels
- Calculate route cost
- Shows road information
- Shows traffic information
- Check road status
- Compares route options

The main menu provides these functions through simple numbered choices.

2.2 How the System Works

1. The system works as follows:
2. The user starts the system using start.
3. The main menu is displayed.
4. The user selects Find Ambulance Route.
5. The user enters the starting location.
6. The user enters the destination.
7. The user selects a route type.
8. The system checks available roads.
9. Blocked roads are avoided.
10. The selected search algorithm finds a route.
11. The result is displayed to the user.

The system can also compare all three route types.

3. Explanation of Code

The Smart Ambulance Route Finding System is developed using Prolog. The system uses facts, predicates, recursion, lists, and search algorithms to find suitable routes.

3.1 Road and Traffic Facts

The road/4 predicate stores road information.

Road(Start, End, Distance, Time).

Example:

road(colombo, kadawatha, 15, 25).

This means there is a road from Colombo to Kadawatha with a distance of 15 km and an estimated travel time of 25 minutes.

This system also stores traffic information using traffic/3.

Traffic(colombo, kadawatha, high).

This means the traffic level on this road is high.

The system uses three traffic levels:

traffic_cost(low, 1).
Traffic_cost(medium, 2).
Traffic_cost(high, 3).

Therefore:

- Low = 1
- Medium = 2
- High = 3

These facts from the main knowledge base of the system.

3.2 Blocked Roads and Road availability

The blocked/2 predicate stores blocked roads.

Example:

blocked(kadawatha, gampaha).

This means the Kadawatha–Gampaha road is blocked.

The available/4 predicate checks whether a road can be used.

```
available(X, Y, Distance, Time) :-
    road(X, Y, Distance, Time),
   \+ blocked(X, Y),
   \+ blocked(Y, X).
```

The system first checks whether the road exists. Then it checks that the road is not blocked.

This allows the route-finding algorithms to avoid blocked roads.

3.3 Road Cost Calculation

The road_cost/3 predicate calculates the cost of using a road.

```
road_cost(X, Y, Cost) :-
    available(X, Y, Distance, _),
    get_traffic(X, Y, Level),
    traffic_cost(Level, TrafficValue),
```

Cost is Distance + TrafficValue * 5.

The formula used is:

Road Cost = Distance + Traffic Value × 5

For example:

Distance = 15 km

Traffic = High = 3

Therefore:

Cost = 15 + (3 × 5) = 30

This road cost is used by the Smart Route algorithm.

3.4 Quick Route - DFS

The Quick Route uses Depth First Search (DFS).

dfs(Start, Goal, Path, Distance).

DFS explores one path deeply before trying another path.

The program keeps track of visited locations using a list. This prevents the system from repeatedly visiting the same location in the current path.

The result contains the selected path and its total distance.

3.5 Shortest Route - BFS

The Shortest Route uses Breadth First Search (BFS).

bfs(Start, Goal, Path, Distance)

BFS explores paths level by level. The program maintains a queue of possible paths and expands them until it finds the destination.

After finding the path, the system calculates its total distance.

This option is used to demonstrate shortest-route searching using BFS.

3.6 Smart Route – A*

The Smart Route uses A* Search.

astar(Start, Goal, Path, Cost)

A* uses the current route cost and a heuristic value.

The calculation is:

F = G + H

Where:

G = current path cost

H = heuristic estimate

F = estimated total cost

The program calculates:

NewG is G + RoadCost,
heuristic(Next, H),
F is NewG + H

The nodes are sorted according to their F value before continuing the search.

3.7 Route Comparison and Main Menu

The compare_routes/2 predicate compares the three route options.
It displays:
Quick Route

- Route
- Distance

Shortest Route

- Route
- Distance

Smart Route

- Route
- Route Cost

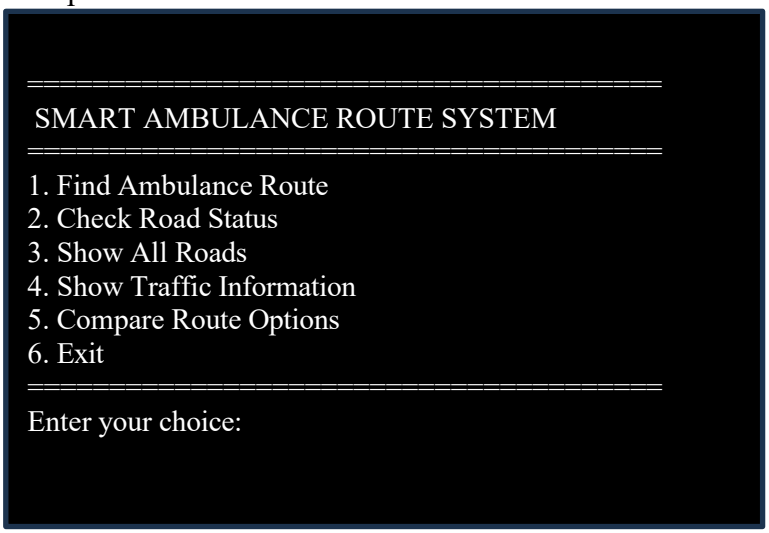
The main menu/0 predicate provides the following options:

1. Find Ambulance Route
2. Check Road Status
3. Show All Roads
4. Show Traffic Information
5. Compare Route Options
6. Exit

The system starts through: **start.**
and then displays the main menu.

3.8 Sample Output

Example 1 – Main Menu

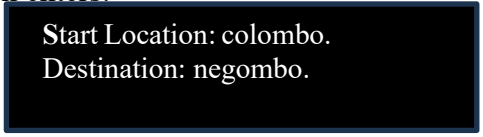


This is the main interface displayed to the ambulance operator.

Example 2 – Find Route
The user selects:

1. Find Ambulance Route

Then enters:



The system displays:

```
Select Route Type:
  1. Quick Route
  2. Shortest Route
  3. Smart Route
  4. Compare Route Options
```

This allows the user to select the required route-finding approach.

Example 3 – Smart Route
When the user selects option 3:

```
SMART ROUTE RESULT
Route: [...]
Route Cost: ...
```

The Smart Route uses road distance together with the traffic-related cost. The defined traffic values are low = 1, medium = 2, and high = 3.

4. Advanced Concepts Used

4.1 Search Algorithms

Search algorithms are important AI techniques used to find a path from a starting state to a goal state.
This project demonstrates three search approaches:

- DFS
- BFS
- A*

Each algorithm has a different searching strategy.

4.2 Depth First Search (DFS)

DFS explores one path deeply before trying another path.
In this project, DFS is used for the Quick Route.
The algorithm keeps track of visited locations to avoid repeated locations in the current path.

4.3 Breadth First Search (BFS)

BFS explores possible paths level by level.
In this project, BFS is used for the Shortest Route.
The program maintains a queue of paths and expands the paths systematically.

4.4 A* Search

A* is an informed search algorithm.
It uses:

$$F = G + H$$

where:
G = current path cost
H = estimated remaining cost
F = total estimated cost
The program calculates F for nodes and sorts the nodes based on this value.

4.5 Heuristic Function

The system uses a heuristic/2 predicate to store estimated values for locations.
For example:

```
heuristic(colombo, 45).
heuristic(kadawatha, 35).
heuristic(gampaha, 25).
heuristic(jaela, 15).
heuristic(negombo, 0).
```

The destination Negombo has a heuristic value of 0.
These values are used by A* when selecting the next node.

4.6 Knowledge Representation

Knowledge representation means storing real-world information in a form that the AI system can process.
In this project, Prolog facts represent:

- Roads
- Distances
- Travel times
- Traffic conditions
- Blocked roads
- Heuristic values

This knowledge is then used by Prolog predicates to make route-finding decisions.

4.7 Recursion and Backtracking

Recursion is used in several parts of the system.
For example, DFS recursively searches from one location to another.
The distance calculation also recursively processes the route until all locations have been handled.
Prolog's logical search and backtracking help the system explore possible routes.

4.8 List Processing

Lists are used to represent paths and queues.
For example:

[Colombo, Kadawatha, Kiribathgoda, Kelaniya]

The BFS implementation uses findall/3 to collect possible new paths before adding them to the queue.
A* also uses lists of nodes and sorts them according to their estimated cost.

4.9 Modular Design

The project follows a modular structure.

Different predicates perform different tasks:

- available/4 → checks road availability
- road_cost/3 → calculates road cost
- dfs/4 → Quick Route
- bfs/4 → Shortest Route
- astar/4 → Smart Route
- check_road/2 → checks road status
- show_roads/0 → displays roads
- show_traffic/0 → displays traffic
- menu/0 → controls user interaction

This makes the system easier to understand and maintain.

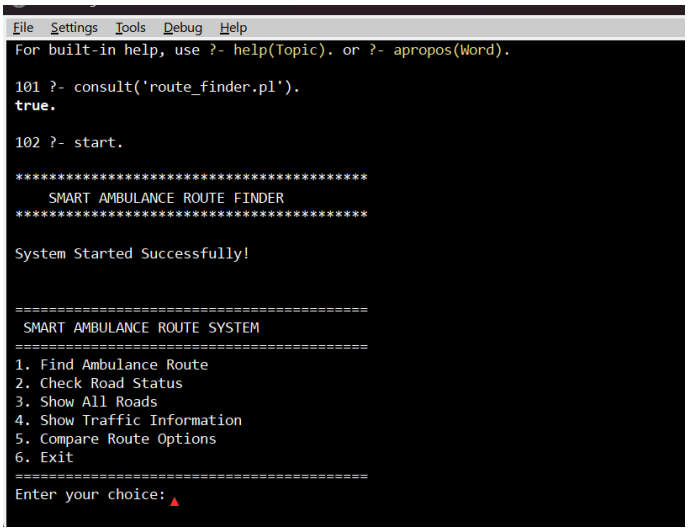
5. Testing and Results

The system was tested using different functions and route options.

Test Case 1 – Main Menu

Result: Main menu display successfully.

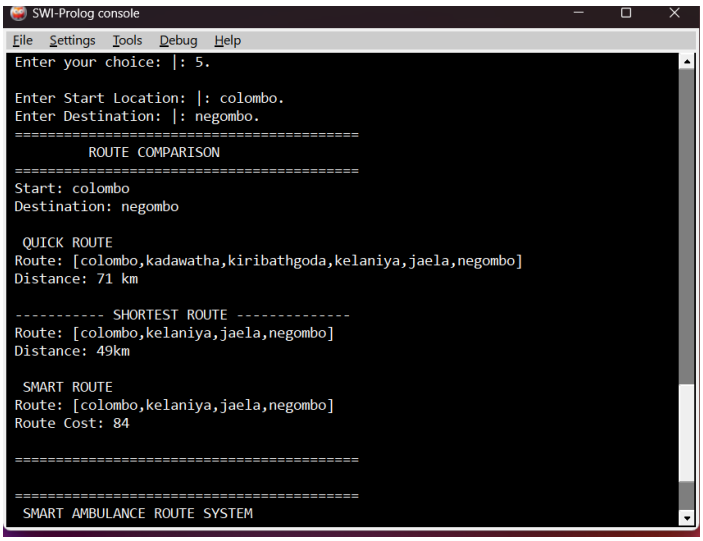
Figure 1:



Test Case 2 – Route Comparison

Result: Quick, Shortest, and Smart routes were displayed

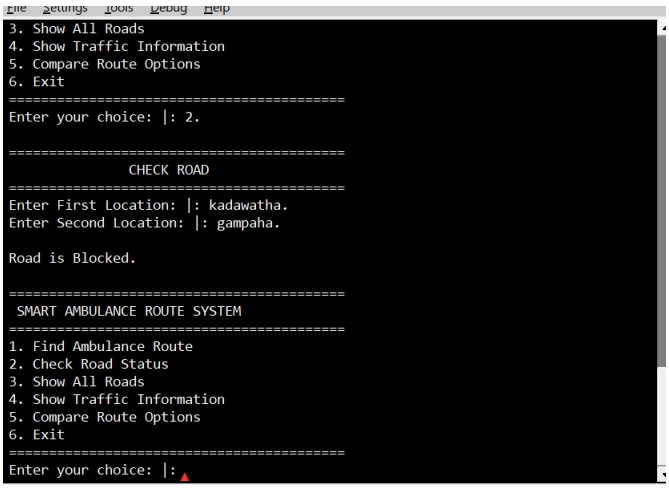
Figure 2:



Test Case 3:

Result: The system correctly displayed “Road is Blocked.”

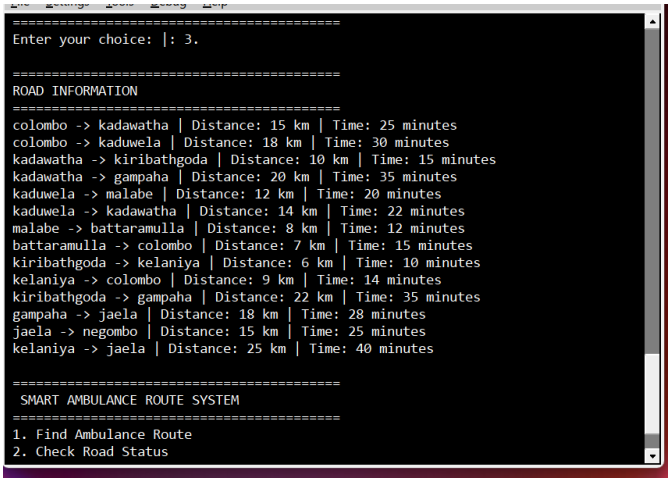
Figure 3:



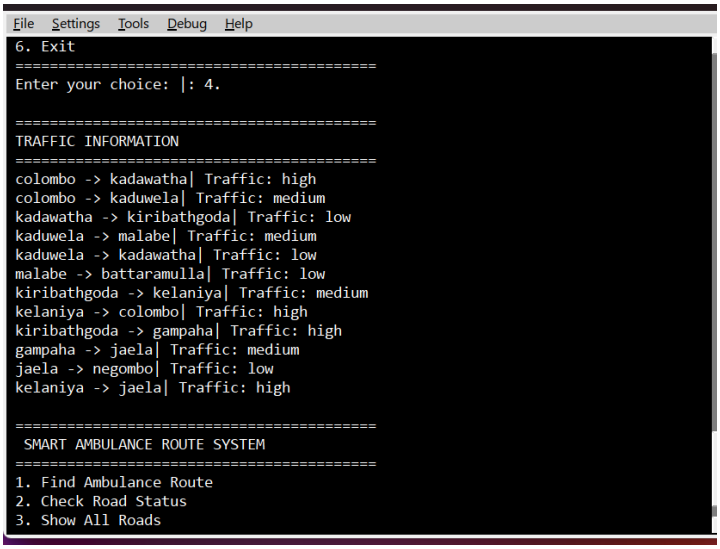
Test Case 4: Road information

Result: All road information was displayed.

Figure 4:



Test Case 5: Traffic information
Result: Traffic information was displayed successfully.
Figure 5:



6. Algorithm Comparison

The three algorithms have different purposes.

Algorithm	System Feature	Main Purpose
DFS	Quick Route	Find a route quickly
BFS	Shortest Route	Search for a shortest route
A*	Smart Route	Find a route using cost and heuristic

The comparison function allows the user to see the results of all three approaches for the same start and destination.
This demonstrates how different AI search algorithms can produce different route-selection behavior.

7. Advantages and Limitations

7.1 Advantages

- Simple and easy-to-use menu system
- Uses Prolog for AI-based reasoning
- Demonstrates three AI search algorithms
- Can avoid blocked roads
- Includes traffic conditions
- Provides route comparison
- Stores road distance and travel time
- Easy to extend with additional road information

7.2 Limitations

- Road information is predefined.
- Traffic information is predefined.
- The system does not use live traffic data.
- The system does not use GPS.
- The number of locations is limited to the knowledge base.
- The current system uses a text-based interface.

8. Future Enhancements

The system can be improved in the future by adding:

- Real-time traffic information
- GPS-based ambulance location
- More cities and roads
- Digital map integration
- Graphical user interface
- Real-time road blockage updates
- Hospital location information
- Hospital availability information
- Multiple ambulance support
- Dynamic route updates

These improvements can make the system more useful in real-world emergency situations.

9. Conclusion

The Smart Ambulance Route Finding System was developed using Prolog and Artificial Intelligence search techniques.

The system represents roads, distances, travel times, traffic conditions, and blocked roads as knowledge. It provides three different route-search methods: Quick Route using DFS, Shortest Route using BFS, and Smart Route using A*.

The system can also check road status, display road and traffic information, and compare different route options.

Therefore, the project demonstrates how AI search algorithms and Prolog knowledge representation can be applied to a real-world ambulance route-finding problem.